Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Методы машинного обучения

ОТЧЕТ

по лабораторной работе №5

Вариант 7

Выполнил
Студент гр. 351002 Михальков А. Д.

Проверил Лебедевич А. В.

Минск 2026

СОДЕРЖАНИЕ

1 Цель выполнения 3
2 Теоретическая основа 3
2.1 Дерево решений как модель классификации 3
2.2 Критерий Джини 3
2.3 Прирост информации 4
2.4 Алгоритм построения дерева 4
2.5 Критерии останова и переобучение 5
2.6 Метрика качества и сравнение с эталонной реализацией 5
3 Индивидуальное задание 6
4 Код решения 7
5 Скриншоты выполнения 13
6 Выводы 16

1 Цель выполнения

Изучение алгоритма построения дерева решений (CART) и приобретение практических навыков его самостоятельной реализации. Реализация критерия Джини, критерия прироста информации, критериев останова и метрики качества модели; построение самописного дерева решений для задачи бинарной классификации на реальном наборе данных и сравнение его результатов с результатами класса DecisionTreeClassifier из библиотеки scikit-learn с проверкой полного совпадения предсказаний.

2 Теоретическая основа

2.1 Дерево решений как модель классификации

Дерево решений – алгоритм машинного обучения, в котором модель представлена иерархической структурой правил вида «если – то». Бинарное дерево состоит из внутренних узлов и листьев (терминальных узлов). В каждом внутреннем узле выбирается признак с номером j и порог t, и объекты делятся на две группы: объекты, для которых $x_j \le t$, переходят в левое (true) поддерево, остальные ($x_j > t$) – в правое (false). Лист хранит обучающие объекты, попавшие в него, и возвращает прогноз – класс, который среди них встречается чаще всего:

$$\hat{y} = \arg\max_k n_k \qquad (1)$$

где n_k – число объектов класса k в листе. Чтобы классифицировать новый объект, его последовательно проверяют условиями узлов, начиная с корня, пока он не достигнет листа.

Деревья решений не требуют масштабирования признаков, описывают нелинейные зависимости и легко интерпретируются. Для их построения используется жадный рекурсивный алгоритм CART (Classification And Regression Trees): в каждом узле выбирается наилучшее по локальному критерию разбиение, после чего процедура повторяется для двух подвыборок.

2.2 Критерий Джини

Качество разбиения оценивается снижением неоднородности (нечистоты) узла. Для классификации широко применяется критерий Джини. Для множества S из l объектов он равен

$$G(S) = 1 - \Sigma\, p_k^2, \quad p_k = n_k / l \qquad (2)$$

где p_k – доля объектов класса k в S. Для чистого узла (все объекты одного класса) G = 0, для двух равновероятных классов G достигает максимума 0,5. Чем меньше G, тем однороднее узел. Альтернативой служит энтропия $H = -\Sigma\, p_k \cdot \log_2 p_k$; в работе используется критерий Джини.

2.3 Прирост информации

Разбиение множества S на левую S_l ($x_j \le t$) и правую S_r ($x_j > t$) части оценивается приростом информации – уменьшением взвешенной неоднородности после разбиения:

$$\mathrm{Gain}(j, t) = G(S) - (l_l/l)\cdot G(S_l) - (l_r/l)\cdot G(S_r) \qquad (3)$$

где l_l и l_r – число объектов в левой и правой частях, $l = l_l + l_r$. Прирост неотрицателен: он равен нулю, если разбиение не повышает однородность (например, оба потомка содержат те же доли классов, что и родитель), и достигает G(S), если оба потомка чистые. В каждом узле выбирается пара «признак – порог», максимизирующая прирост:

$$(j^*, t^*) = \arg\max \mathrm{Gain}(j, t) \qquad (4)$$

2.4 Алгоритм построения дерева

Дерево строится рекурсивно. Для текущего узла сначала проверяются критерии останова, и если выполнен хотя бы один из них, создаётся лист. В противном случае перебираются все признаки и все допустимые пороги, для каждой пары вычисляется прирост информации (3), выбирается пара (j^*, t^*) с максимальным приростом, выборка делится по правилу $x_j \le t$, и алгоритм рекурсивно применяется к левой и правой частям.

Кандидатами в пороги для непрерывного признака служат середины между соседними различными значениями признака в узле:

$$t = (v_i + v_{i+1}) / 2 \qquad (5)$$

где $v_1 < v_2 < \ldots$ – отсортированные уникальные значения признака. Любой порог между v_i и v_{i+1} даёт одно и то же деление выборки, поэтому достаточно проверить по одному порогу на интервал; именно такие пороги использует sklearn. Наивный перебор требует для каждого порога заново вычислять критерий по всем объектам узла, поэтому его сложность растёт примерно как $O(d \cdot l^2)$ для d признаков и l объектов, тогда как в sklearn применяются предварительная сортировка и инкрементальный пересчёт. Поэтому самописная реализация заметно медленнее эталонной.

2.5 Критерии останова и переобучение

Дерево без ограничений делит выборку до тех пор, пока все листья не станут чистыми, то есть запоминает обучающую выборку (переобучение) и плохо обобщается на новые данные. Поэтому рост дерева ограничивают критериями останова:

– максимальная глубина дерева max_depth: узел на глубине max_depth становится листом;

– минимальное число объектов в узле min_samples_split: узел с меньшим числом объектов не делится;

– минимальное число объектов в листе min_samples_leaf: допускаются только разбиения, после которых в каждом потомке не меньше заданного числа объектов;

– чистота узла и отсутствие прироста: если $G(S) = 0$ или ни одно допустимое разбиение не даёт положительного прироста, узел становится листом.

Чем жёстче ограничения, тем проще дерево и выше риск недообучения; чем они мягче, тем выше риск переобучения.

2.6 Метрика качества и сравнение с эталонной реализацией

Качество классификации оценивается долей правильных ответов (Accuracy):

$$\text{Accuracy} = (1/l)\cdot\Sigma\, I[a(x_i) = y_i] \qquad (6)$$

где $a(x_i)$ – прогноз модели, y_i – истинная метка, $I[\cdot]$ – индикатор события. Для проверки корректности самописного дерева его результаты сравниваются с DecisionTreeClassifier из библиотеки scikit-learn, реализующим алгоритм CART (criterion='gini'). Чтобы результаты совпали полностью, необходимо использовать те же правила: одинаковые кандидаты порогов (середины между соседними значениями), условие ветвления $x_j \le t$, те же значения критериев останова, а в листе при равенстве числа объектов разных классов выбирать класс с меньшей меткой. Единственным источником расхождений остаётся случайный выбор sklearn среди разбиений с одинаково высоким приростом (зависит от random_state). Полное совпадение проверяется поэлементным сравнением предсказаний.

3 Индивидуальное задание

В рамках лабораторной работы необходимо на основе приведённого в задании кода (классы Node и Leaf, функции split, find_best_split, build_tree, classify_object, predict, print_tree) выполнить следующие этапы:

1) Реализовать функцию расчёта критерия Джини.
2) Реализовать расчёт критерия прироста информации.
3) Реализовать критерии останова (не менее двух).
4) Реализовать функцию подсчёта метрики модели.
5) Проверить работу самописного дерева для решения задачи классификации (набор данных можно сгенерировать самостоятельно функцией make_classification либо взять любой набор данных из задания к ЛР4; в работе использован Heart_Disease_and_Hospitals.csv, целевой признак heart_disease), сравнив полученные результаты с результатами дерева решений из sklearn. Результаты должны совпасть полностью.

4 Код решения

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('Heart_Disease_and_Hospitals.csv')
print(f"Исходный размер таблицы: {df.shape[0]} строк, {df.shape[1]} колонок")

feature_cols = ['age', 'blood_pressure', 'cholesterol', 'bmi', 'glucose_level']
target_col = 'heart_disease'

clean_df = df[feature_cols + [target_col]].dropna().reset_index(drop=True)

X = clean_df[feature_cols].values.astype(float)
y = clean_df[target_col].values.astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print(f"Данные готовы к работе:")
print(f"Обучающая выборка (пациентов): {X_train.shape[0]}")
print(f"Тестовая выборка (пациентов): {X_test.shape[0]}")
print(f"Признаки: {feature_cols}")
print(f"Распределение классов (болен/здоров): {np.bincount(y)}")

class Node:
    """Класс узла дерева, содержащий правило разбиения."""
    def __init__(self, index, t, true_branch, false_branch):
        self.index = index
        self.t = t
        self.true_branch = true_branch
        self.false_branch = false_branch

class Leaf:
    """Класс терминального листа дерева, возвращающий предсказание."""
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels
        self.prediction = self.predict()

    def predict(self):
        classes = {}
        for label in self.labels:
            if label not in classes:
                classes[label] = 0
            classes[label] += 1

        return max(sorted(classes), key=classes.get)

print("Базовые классы Node и Leaf успешно определены.")

def gini(labels):
    """
    Расчёт критерия неопределённости Джини для массива меток.

    Параметры:
        labels: массив или список меток целевой переменной (классов)
    Возвращает:
        float: значение критерия Джини от 0.0 (чистый узел) до 0.5 (максимальный хаос)
    """
    if len(labels) == 0:
        return 0.0
```

```
    classes = {}
    for label in labels:
        if label not in classes:
            classes[label] = 0
        classes[label] += 1

    impurity = 1.0
    total_objects = len(labels)

    for count in classes.values():
        prob = count / total_objects
        impurity -= prob ** 2

    return impurity

test_1 = [0, 0, 1, 1]
test_2 = [1, 1, 1, 1]
test_3 = [0, 0, 0, 1]

print(f"Тест 1 (50/50):           Gini = {gini(test_1):.4f} (Ожидается: 0.5000)")
print(f"Тест 2 (Все одного класса): Gini = {gini(test_2):.4f} (Ожидается: 0.0000)")
print(f"Тест 3 (Небольшой перекос): Gini = {gini(test_3):.4f} (Ожидается: 0.3750)")

def split(data, labels, column_index, t):
    """
    Разделяет данные и метки на две ветви по условию: признак <= t.
    """
    left_mask = data[:, column_index] <= t
    right_mask = data[:, column_index] > t

    true_data, false_data = data[left_mask], data[right_mask]
    true_labels, false_labels = labels[left_mask], labels[right_mask]

    return true_data, false_data, true_labels, false_labels


def gain(left_labels, right_labels, root_gini):
    """
    Расчёт прироста информации (Information Gain).

    Параметры:
        left_labels: метки объектов, ушедших в левую ветку (True)
        right_labels: метки объектов, ушедших в правую ветку (False)
        root_gini: значение критерия Джини исходного узла до разбиения
    """
    n_left = len(left_labels)
    n_right = len(right_labels)
    n_total = n_left + n_right

    if n_total == 0 or n_left == 0 or n_right == 0:
        return 0.0

    p_left = n_left / n_total
    p_right = n_right / n_total

    weighted_gini = p_left * gini(left_labels) + p_right * gini(right_labels)

    return root_gini - weighted_gini

root = [0, 0, 0, 1, 1, 1]
root_g = gini(root)

left_perfect = [0, 0, 0]
right_perfect = [1, 1, 1]
gain_perfect = gain(left_perfect, right_perfect, root_g)

left_useless = [0, 1]
right_useless = [0, 0, 1, 1]
```

```
gain_useless = gain(left_useless, right_useless, root_g)

print(f"Исходный Gini родителя: {root_g:.4f}")
print(f"Прирост при идеальном разбиении:  Gain = {gain_perfect:.4f} (Ожидается: 0.5000)")
print(f"Прирост при бесполезном разбиении: Gain = {gain_useless:.4f} (Ожидается: 0.0000)")

def find_best_split(data, labels, min_samples_leaf=1):
    """
    Поиск наилучшего признака и порога для разбиения узла.
    Учитывает критерий останова: min_samples_leaf.
    """
    root_gini = gini(labels)
    best_gain = 0.0
    best_t = None
    best_index = None

    n_features = data.shape[1]

    for index in range(n_features):
        values = np.unique(data[:, index])

        t_values = (values[:-1] + values[1:]) / 2.0

        for t in t_values:
            true_data, false_data, true_labels, false_labels = split(data, labels, index, t)

            if len(true_labels) < min_samples_leaf or len(false_labels) < min_samples_leaf:
                continue

            current_gain = gain(true_labels, false_labels, root_gini)

            if current_gain > best_gain + 1e-12:
                best_gain = current_gain
                best_t = t
                best_index = index

    return best_gain, best_t, best_index


def build_tree(data, labels, max_depth=None, min_samples_split=2, min_samples_leaf=1,
depth=0):
    """
    Рекурсивная функция построения решающего дерева.
    """
    if max_depth is not None and depth >= max_depth:
        return Leaf(data, labels)

    if len(labels) < min_samples_split:
        return Leaf(data, labels)

    if gini(labels) == 0.0:
        return Leaf(data, labels)

    best_gain, t, index = find_best_split(data, labels, min_samples_leaf)

    if best_gain <= 0.0 or t is None:
        return Leaf(data, labels)

    true_data, false_data, true_labels, false_labels = split(data, labels, index, t)

    true_branch = build_tree(true_data, true_labels, max_depth, min_samples_split,
min_samples_leaf, depth + 1)
    false_branch = build_tree(false_data, false_labels, max_depth, min_samples_split,
min_samples_leaf, depth + 1)

    return Node(index, t, true_branch, false_branch)

print("Функции find_best_split и build_tree готовы.")
```

```
def classify_object(obj, node):
    """
    Рекурсивно спускает один объект по дереву решений до листа и возвращает диагноз.
    """
    if isinstance(node, Leaf):
        return node.prediction

    if obj[node.index] <= node.t:
        return classify_object(obj, node.true_branch)
    else:
        return classify_object(obj, node.false_branch)


def predict(data, tree):
    """
    Применяет дерево ко всему набору данных (массиву объектов)
    и возвращает список предсказаний.
    """
    return [classify_object(obj, tree) for obj in data]


def print_tree(node, spacing=""):
    """
    Рекурсивно распечатывает структуру дерева в читаемом текстовом виде.
    """
    if isinstance(node, Leaf):
        print(spacing + "Прогноз:", node.prediction)
        return

    feature_name = feature_cols[node.index] if 'feature_cols' in globals() else f"Признак
{node.index}"
    print(spacing + f"{feature_name} (индекс {node.index}) <= {node.t:.4f}")

    print(spacing + '--> True:')
    print_tree(node.true_branch, spacing + "  ")

    print(spacing + '--> False:')
    print_tree(node.false_branch, spacing + "  ")

print("Функции classify_object, predict и print_tree успешно добавлены.")

def accuracy_metric(actual, predicted):
    """
    Вычисляет долю правильных ответов (Accuracy).

    Параметры:
        actual: истинные метки классов (массив или список)
        predicted: предсказанные метки классов (массив или список)
    Возвращает:
        float: значение точности от 0.0 до 1.0
    """
    correct = 0
    total = len(actual)

    if total == 0:
        return 0.0

    for i in range(total):
        if actual[i] == predicted[i]:
            correct += 1

    return correct / float(total)

test_true = [0, 1, 0, 1]
test_pred = [0, 1, 1, 1]

acc_custom = accuracy_metric(test_true, test_pred)
```

```
acc_sklearn = accuracy_score(test_true, test_pred)

print(f"Наша функция accuracy:   {acc_custom:.4f}")
print(f"Функция из sklearn:      {acc_sklearn:.4f}")
print(f"Результаты совпали:      {acc_custom == acc_sklearn}")

X_sub = X[:1000]
y_sub = y[:1000]

X_train_sub, X_test_sub, y_train_sub, y_test_sub = train_test_split(
    X_sub, y_sub, test_size=0.3, random_state=42, stratify=y_sub
)

params = dict(max_depth=4, min_samples_split=10, min_samples_leaf=5)

my_tree = build_tree(X_train_sub, y_train_sub, **params)

sk_tree = DecisionTreeClassifier(criterion='gini', random_state=42, **params)
sk_tree.fit(X_train_sub, y_train_sub)

my_train_pred = np.array(predict(X_train_sub, my_tree))
my_test_pred = np.array(predict(X_test_sub, my_tree))

sk_train_pred = sk_tree.predict(X_train_sub)
sk_test_pred = sk_tree.predict(X_test_sub)

my_acc_train = accuracy_metric(y_train_sub, my_train_pred)
my_acc_test = accuracy_metric(y_test_sub, my_test_pred)

sk_acc_train = accuracy_score(y_train_sub, sk_train_pred)
sk_acc_test = accuracy_score(y_test_sub, sk_test_pred)

print("=" * 60)
print("РЕЗУЛЬТАТЫ СРАВНЕНИЯ МОДЕЛЕЙ")
print("=" * 60)
print(f"Самописное дерево:")
print(f"  Accuracy на обучении: {my_acc_train:.6f}")
print(f"  Accuracy на тесте:    {my_acc_test:.6f}")
print(f"\nМодель sklearn:")
print(f"  Accuracy на обучении: {sk_acc_train:.6f}")
print(f"  Accuracy на тесте:    {sk_acc_test:.6f}")

print("\n" + "-" * 60)
print("ПРОВЕРКА НА ПОЛНОЕ СОВПАДЕНИЕ:")
train_match = np.array_equal(my_train_pred, sk_train_pred)
test_match = np.array_equal(my_test_pred, sk_test_pred)
print(f"Предсказания на train совпали на 100%: {train_match}")
print(f"Предсказания на test совпали на 100%:  {test_match}")
print("-" * 60)

print("\nСтруктура первых уровней построенного дерева:")
print_tree(my_tree)

import matplotlib.pyplot as plt

importances = sk_tree.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(9, 5))
plt.title("Важность признаков в диагностике заболеваний сердца", fontsize=14)
plt.barh(range(len(indices)), importances[indices], color='skyblue', edgecolor='navy')
plt.yticks(range(len(indices)), [feature_cols[i] for i in indices], fontsize=11)
plt.xlabel("Относительная важность (Information Gain contribution)", fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

for i in reversed(indices):
```

```
    print(f"Признак '{feature_cols[i]}': важность = {importances[i]:.4f}")

experiments = [
    {"max_depth": 2, "min_samples_split": 2,  "min_samples_leaf": 1},
    {"max_depth": 3, "min_samples_split": 5,  "min_samples_leaf": 2},
    {"max_depth": 4, "min_samples_split": 10, "min_samples_leaf": 5},
    {"max_depth": 5, "min_samples_split": 10, "min_samples_leaf": 5},
    {"max_depth": 6, "min_samples_split": 15, "min_samples_leaf": 8},
]

table_rows = []

for exp in experiments:
    t_my = build_tree(X_train_sub, y_train_sub, **exp)
    pred_my = predict(X_test_sub, t_my)
    acc_my = accuracy_metric(y_test_sub, pred_my)

    t_sk = DecisionTreeClassifier(criterion='gini', random_state=42, **exp)
    t_sk.fit(X_train_sub, y_train_sub)
    pred_sk = t_sk.predict(X_test_sub)
    acc_sk = accuracy_score(y_test_sub, pred_sk)

    table_rows.append({
        "max_depth": exp["max_depth"],
        "min_samples_split": exp["min_samples_split"],
        "min_samples_leaf": exp["min_samples_leaf"],
        "Accuracy (Наше)": round(acc_my, 4),
        "Accuracy (sklearn)": round(acc_sk, 4),
        "100% совпадение": np.array_equal(pred_my, pred_sk)
    })

results_df = pd.DataFrame(table_rows)
display(results_df)
```

5 Скриншоты выполнения

```
Исходный размер таблицы: 10000 строк, 15 колонок
Данные готовы к работе:
Обучающая выборка (пациентов): 7000
Тестовая выборка (пациентов): 3000
Признаки: ['age', 'blood_pressure', 'cholesterol', 'bmi', 'glucose_level']
Распределение классов (болен/здоров): [4993 5007]
```

Рисунок 1 – Загрузка и подготовка датасета Heart_Disease_and_Hospitals.csv

```
Базовые классы Node и Leaf успешно определены.

Функции find_best_split и build_tree готовы.

Функции classify_object, predict и print_tree успешно добавлены.
```

Рисунок 2 – Сообщения об определении классов Node, Leaf и функций построения и применения дерева

```
Тест 1 (50/50):           Gini = 0.5000 (Ожидается: 0.5000)
Тест 2 (Все одного класса): Gini = 0.0000 (Ожидается: 0.0000)
Тест 3 (Небольшой перекос): Gini = 0.3750 (Ожидается: 0.3750)
```

Рисунок 3 – Проверка функции расчёта критерия Джини

```
Исходный Gini родителя: 0.5000
Прирост при идеальном разбиении:  Gain = 0.5000 (Ожидается: 0.5000)
Прирост при бесполезном разбиении: Gain = 0.0000 (Ожидается: 0.0000)
```

Рисунок 4 – Проверка функции расчёта прироста информации

```
Наша функция accuracy:   0.7500
Функция из sklearn:      0.7500
Результаты совпали:      True
```

Рисунок 5 – Проверка функции расчёта метрики Accuracy

```
============================================================
РЕЗУЛЬТАТЫ СРАВНЕНИЯ МОДЕЛЕЙ
============================================================
Самописное дерево:
  Accuracy на обучении: 0.898571
  Accuracy на тесте:    0.870000

Модель sklearn:
  Accuracy на обучении: 0.898571
  Accuracy на тесте:    0.870000

------------------------------------------------------------
ПРОВЕРКА НА ПОЛНОЕ СОВПАДЕНИЕ:
Предсказания на train совпали на 100%: True
Предсказания на test совпали на 100%:  True
------------------------------------------------------------
```

Рисунок 6 – Сравнение самописного дерева с моделью sklearn

```
Структура первых уровней построенного дерева:
bmi (индекс 3) <= 29.5403
--> True:
  glucose_level (индекс 4) <= 95.8069
  --> True:
    blood_pressure (индекс 1) <= 131.0825
    --> True:
      glucose_level (индекс 4) <= 57.0467
      --> True:
        Прогноз: 0
      --> False:
        Прогноз: 0
    --> False:
      cholesterol (индекс 2) <= 165.2610
      --> True:
        Прогноз: 0
      --> False:
        Прогноз: 1
  --> False:
    cholesterol (индекс 2) <= 197.7315
    --> True:
      cholesterol (индекс 2) <= 36.1473
      --> True:
        Прогноз: 0
      --> False:
        Прогноз: 1
    --> False:
      cholesterol (индекс 2) <= 250.4415
      --> True:
        Прогноз: 0
      --> False:
        Прогноз: 0
--> False:
  blood_pressure (индекс 1) <= 65.9820
  --> True:
    Прогноз: 0
  --> False:
    blood_pressure (индекс 1) <= 85.7208
    --> True:
      age (индекс 0) <= 62.5000
      --> True:
        Прогноз: 1
      --> False:
        Прогноз: 1
    --> False:
      glucose_level (индекс 4) <= -37.1719
      --> True:
        Прогноз: 1
      --> False:
        Прогноз: 1
```

Рисунок 7 – Структура построенного самописного дерева решений (слева – ветвь True корня, справа – ветвь False)

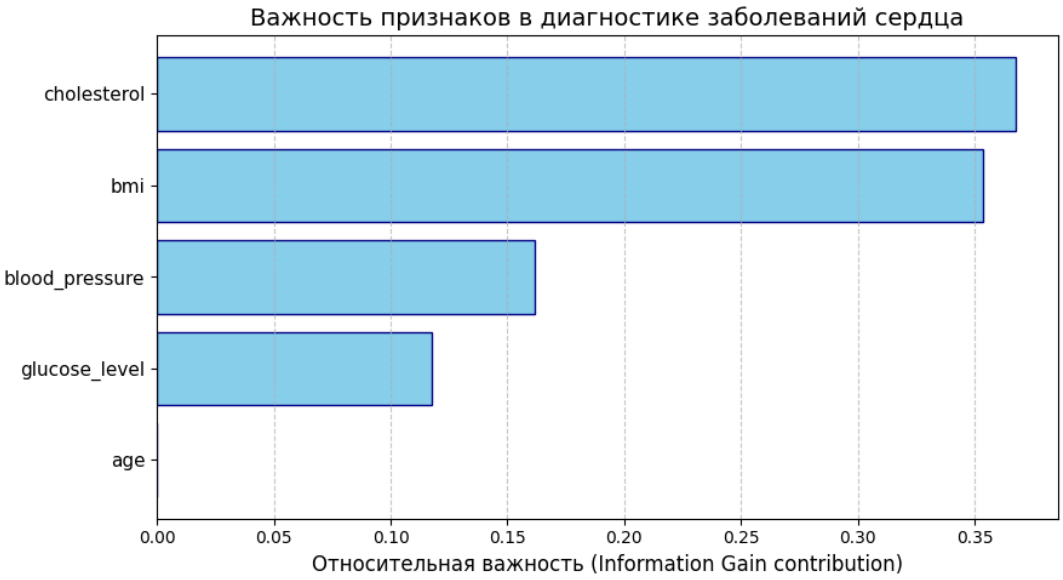


Рисунок 8 – Важность признаков в дереве sklearn

```
Признак 'cholesterol': важность = 0.3673
Признак 'bmi': важность = 0.3537
Признак 'blood_pressure': важность = 0.1615
Признак 'glucose_level': важность = 0.1175
Признак 'age': важность = 0.0000
```

Рисунок 9 – Числовые значения важности признаков

	max_depth	min_samples_split	min_samples_leaf	Accuracy (Наше)	Accuracy (sklearn)	100% совпадение
0	2	2	1	0.7133	0.7133	True
1	3	5	2	0.8867	0.8867	True
2	4	10	5	0.8700	0.8700	True
3	5	10	5	0.8667	0.8667	True
4	6	15	8	0.8833	0.8833	True

Рисунок 10 – Влияние гиперпараметров дерева на точность и совпадение предсказаний с sklearn

6 Выводы

В ходе выполнения лабораторной работы был реализован алгоритм построения дерева решений для задачи бинарной классификации (определение наличия сердечного заболевания), а его результаты сопоставлены с моделью DecisionTreeClassifier из библиотеки scikit-learn. На основании полученных результатов сделаны следующие выводы:

1) Реализованы функция расчёта критерия Джини, функция прироста информации и метрика Accuracy; они проверены на контрольных примерах: Gini([0, 0, 1, 1]) = 0,5, Gini([1, 1, 1, 1]) = 0, Gini([0, 0, 0, 1]) = 0,375; прирост равен 0,5 для идеального разбиения и 0 для бесполезного; Accuracy на тестовом примере (0,75) совпала со значением accuracy_score из sklearn.
2) Для проверки использован набор данных Heart_Disease_and_Hospitals.csv (10 000 записей, 15 столбцов). Признаками выбраны age, blood_pressure, cholesterol, bmi и glucose_level, целевой признак – heart_disease. Пропусков в выбранных столбцах нет, классы сбалансированы (4993 здоровых и 5007 больных). Выборка разделена на обучающую (7000) и тестовую (3000) части со стратификацией. Сравнение с sklearn выполнено на подвыборке из первых 1000 объектов (700 обучающих и 300 тестовых), так как наивный перебор порогов в самописной реализации имеет квадратичную сложность по числу объектов.
3) Реализованы четыре критерия останова: максимальная глубина дерева (max_depth), минимальное число объектов для разбиения узла (min_samples_split), минимальное число объектов в листе (min_samples_leaf) и остановка при чистом узле либо отсутствии прироста. Для полного совпадения с sklearn пороги берутся как середины между соседними уникальными значениями признака, а в листе при равенстве классов выбирается меньшая метка.
4) При параметрах max_depth = 4, min_samples_split = 10, min_samples_leaf = 5 самописное дерево и DecisionTreeClassifier показали одинаковую точность: 0,898571 на обучающей и 0,870000 на тестовой выборке; предсказания совпали на 100 % как на обучающей, так и на тестовой выборках.
5) Для пяти наборов гиперпараметров (глубина от 2 до 6) точность самописного дерева и дерева sklearn совпала до четвёртого знака, предсказания на тесте совпали полностью. Глубина 2 даёт недообучение (0,7133), при глубинах 3–6 точность на тесте находится

в диапазоне 0,8667–0,8867; разница между ними составляет не более 6 объектов из 300 и сопоставима с шумом на небольшой тестовой выборке, поэтому дальнейшее увеличение глубины не оправдано.

6) По важности признаков в построенном дереве лидируют cholesterol (0,3673) и bmi (0,3537), далее идут blood_pressure (0,1615) и glucose_level (0,1175); признак age в разбиениях не используется (важность 0). Следует учитывать, что данные не очищались от аномальных значений: в структуре дерева встречается порог glucose_level $\leq -37{,}17$, то есть набор содержит физически невозможные значения, что для дерева решений не нарушает работу алгоритма, но снижает смысловую ценность отдельных правил.
7) Таким образом, самописная реализация дерева решений корректна: при одинаковых правилах выбора порогов, критериях останова и разрешении равенств она воспроизводит результаты sklearn полностью. Ограничениями работы являются использование одного разбиения на обучающую и тестовую выборки с фиксированным random_state и меньшая скорость самописного алгоритма по сравнению с эталонным.